certmagic `selectPreferredChain` empty-chain panic — Root-Cause Analysis (upstream bug #354)

Revision: 1 **Last modified:** 2026-07-01T10:06:36Z **Authority:** Helix Constitution §11.4.102 (systematic-debugging: NO fixes without root-cause investigation first) + §11.4.114 (last-known-good-tag regression isolation) + §11.4.6 (no-guessing — every claim cited to source or marked UNCONFIRMED) + §11.4.150 (deep multi-angle research per issue). **Scope:** the `runtime error: index out of range [0] with length 0` panic hit inside Caddy's in-process ACME client (certmagic) during the hermetic Let's Encrypt issuance phase (`deploy/letsencrypt/phase3_hermetic_issue.sh`), its FACT-grade root cause, the exact vulnerable/fixed versions, and the ranked fix.

> **Method note (§11.4.6):** the panic site, the loop structure, and the fixed-version boundary below are FACT — read directly from certmagic source at the pinned tags. Where a runtime detail (which of the two attempts failed first, the exact Pebble restart timing) could not be pinned from a log, it is stated as the observed trigger, not as the mechanism. The mechanism is source-proven.

---

1. The panic (symptom)

```
panic: runtime error: index out of range [0] with length 0

goroutine ... [running]:
github.com/caddyserver/certmagic.
(*ACMEIssuer).selectPreferredChain(...)
    .../certmagic@v0.21.3/acmeissuer.go:592
github.com/caddyserver/certmagic.(*ACMEIssuer).doIssue(...)
    .../certmagic@v0.21.3/acmeissuer.go:511
```

`selectPreferredChain` dereferences `certChains[0]` with **no length guard**. When it is handed an empty (`nil` / `len==0`) slice of certificate chains, the `[0]` index panics and takes the whole Caddy process down (an unrecovered panic in the ACME goroutine).

This is upstream **certmagic issue #354** ("panic: runtime error: index out of range [0] with length 0" in `selectPreferredChain`), fixed by commit **`ecf3f80`**, first released in certmagic **v0.25.1** (first Caddy release carrying that certmagic: **Caddy 2.11.0**).

---

2. Root cause (FACT — source-proven)

The bug is in `(*ACMEIssuer).doIssue` in `acmeissuer.go`. Structurally (v0.21.3):

```
func (iss *ACMEIssuer) doIssue(ctx context.Context, csr
        *x509.CertificateRequest, useTestCA bool)
        (*IssuedCertificate, bool, error) {
    ...
    var certChains []acme.Certificate
    for i := 0; i < 2; i++
        {                  // <= retries AT MOST twice
        certChains, err =
        client.acmeClient.ObtainCertificate(ctx, account,
        csr, ...)
        if err == nil {
            // empty-chain guard lives ONLY on the success branch:
            if len(certChains) == 0 {
                return nil, false, fmt.Errorf("no certificate
        chains")
            }
            break
        }
        // error branch — classify the ACME problem:
        var problem acme.Problem
        if errors.As(err, &problem) {
            if problem.Type ==
        acme.ProblemTypeAccountDoesNotExist {
                // the local account is stale: recreate it and
        RETRY
                resetAccount(...)
                continue                    // <= loops WITHOUT
        populating certChains
            }
        }
        return nil, false, err
    }

    // line 511 — reached after the loop with certChains possibly
        still nil:
```

```
    preferredChain, err :=
        iss.selectPreferredChain(certChains)   // panics if
        certChains == nil
    ...
}
```

The defect is the **placement of the empty-chain guard**: `if len(certChains) == 0 { ... }` is INSIDE the loop and ONLY on the `err == nil` branch. The `ProblemTypeAccountDoesNotExist` path does `continue` **without** ever assigning a non-empty `certChains`.

The fatal interleaving: the loop runs at most twice (`for i := 0; i < 2; i++`). If **BOTH** iterations hit `ProblemTypeAccountDoesNotExist` and `continue`, the loop condition falls through with `certChains` still `nil`. Execution reaches line 511, which calls `selectPreferredChain(nil)`; that function indexes `certChains[0]` (acmeissuer.go:592) → `index out of range [0] with length 0` → panic.

In other words: the "account does not exist, recreate and retry" recovery path had no terminal guard for the case where *every* retry is consumed by that same recovery — so instead of returning a clean, retryable error it fell into an unguarded index and crashed the process. The upstream fix (`ecf3f80`) adds the missing guard so an exhausted retry loop returns an error rather than panicking on an empty chain.

---

3. Why the hermetic test triggered it (observed trigger)

The panic is latent in normal operation and surfaces only when both retries are consumed by `accountDoesNotExist`. The hermetic harness manufactured exactly that condition:

1. **Pebble is in-memory.** Pebble does not persist accounts — a restart discards all registered ACME accounts (this is the same non-volatile-storage property documented in the sibling `letsencrypt_hermetic_20260701/ANALYSIS.md` Q1). Any Caddy account registered against a previous Pebble process instantly becomes "does not exist" to a fresh Pebble.
2. **Service-recreate churn.** Repeated boot/teardown of the compose stack recreated Pebble underneath a Caddy that still held a cached account handle → the CA answered `ProblemTypeAccountDoesNotExist` for that stale account.
3. **A transient `dial tcp :14000: connection refused`.** Caddy racing Pebble's readiness (Caddy started before Pebble's ACME listener was accepting) produced a connection failure on one attempt; combined with the stale-account error on the other, **both** loop iterations failed via the `accountDoesNotExist` classification and `continued` — exhausting the two-iteration

budget with `certChains == nil`, driving the code straight into the unguarded `selectPreferredChain(nil)`.

So the harness's boot ordering + Pebble's volatility manufactured the double-`accountDoesNotExist` interleaving that the code could not survive.

---

4. Vulnerable vs fixed certmagic versions (FACT)

certmagic version	Panic present?
v0.21.3	**VULNERABLE** (panic site as quoted above)
v0.21.4	**VULNERABLE**
v0.21.6	**VULNERABLE**
v0.21.7	**VULNERABLE**
v0.23.0	**VULNERABLE**
v0.24.0	**VULNERABLE**
v0.25.0	**VULNERABLE**
v0.25.1	**FIXED** (commit `ecf3f80`, issue #354)
>= v0.25.1	FIXED

Caddy pinning (from Caddy `go.mod`): **Caddy 2.8.4** pins a vulnerable certmagic (v0.21.x line); **Caddy 2.11.0** is the first Caddy release whose `go.mod` pins certmagic **>= v0.25.1**, i.e. the first Caddy that carries the fix. See Caddy issue **#7366** (the same empty-chain panic reported against Caddy) and certmagic issue **#152** (Pebble per-launch CA regeneration — the volatility that feeds the trigger).

---

5. Fix (ranked)

Rank 1 — clean boot ordering (the harness fix already applied). The panic requires the *double-`accountDoesNotExist`* interleaving. Remove the interleaving and the vulnerable code path is never entered even on the vulnerable certmagic:

- Bring **Pebble up and HEALTHY before Caddy** so Caddy never races an absent ACME listener (kills the `dial tcp :14000 connection refused` attempt).

- Give Caddy a **fresh `/data` (fresh storage) per run** so no stale account handle from a previous Pebble is cached (kills the stale-account `accountDoesNotExist`).
- **Do not churn Pebble mid-order** — one Pebble process spans the whole issuance so accounts stay valid for the order's lifetime.

This is precisely what `deploy/letsencrypt/phase3_hermetic_issue.sh` now does, and under that ordering it issues a **real certificate with no panic** (evidence: the conductor's clean run under `qa-results/letsencrypt/phase3_issuance/`).

Rank 2 — bump to Caddy >= 2.11.0 / certmagic >= v0.25.1. The upstream fix converts the crash into a **retryable error** (an exhausted account-recreate loop returns `error`, not a panic). This is defence-in-depth: even if a future ordering regression reintroduces the double-`accountDoesNotExist` interleaving, Caddy survives with a logged, retried error instead of a process-killing panic.

Best — do both. Rank 1 removes the trigger for *this* harness now (independent of the certmagic version we happen to build against); Rank 2 removes the crash mode structurally for every future ordering. Rank 1 alone is sufficient to make the current hermetic phase green; Rank 2 alone (without clean ordering) would still churn through retries and re-issue noise on every restart. Together: no trigger AND no crash mode.

---

6. Honest boundary (§11.4.6)

- **FACT (source-proven):** the panic site (`acmeissuer.go:592`, `selectPreferredChain` indexing `certChains[0]`), the `doIssue` two-iteration loop with the guard only on the `err==nil` branch, the `ProblemTypeAccountDoesNotExist` `continue`-without-populating path, the vulnerable-version list, and the fixed boundary at certmagic **v0.25.1** / Caddy **2.11.0** (commit `ecf3f80`, issue #354) — all read from certmagic source at the pinned tags and Caddy's `go.mod`.
- **Observed (not mechanism):** the exact ordering of which of the two attempts hit `connection refused` vs stale-account in the failing run was inferred from the boot sequence + Pebble's known volatility, not pinned from a captured per-attempt log line. The *mechanism* (double-`accountDoesNotExist` → `nil` chain → panic) is source-proven and version-independent; the specific transient that produced the second `accountDoesNotExist` is the observed trigger.
- **PROVEN for THIS harness:** that Rank-1 clean ordering ALONE fixes the panic here is proven by the conductor's clean run — a real certificate was issued with no panic (`qa-results/letsencrypt/phase3_issuance/`). This does not claim Rank 1

removes the *code-level* crash mode (only Rank 2 does); it claims Rank 1 removes the *trigger* for this harness, which the clean run demonstrates.

Sources verified 2026-07-01

- certmagic `acmeissuer.go` at **v0.21.3** (vulnerable — panic site line 592, `doIssue` loop) — https://raw.githubusercontent.com/caddyserver/certmagic/v0.21.3/acmeissuer.go
- certmagic `acmeissuer.go` at **v0.25.1** (fixed — guard added) — https://raw.githubusercontent.com/caddyserver/certmagic/v0.25.1/acmeissuer.go
- certmagic issue **#354** — "panic: runtime error: index out of range [0] with length 0" (`selectPreferredChain`) — https://github.com/caddyserver/certmagic/issues/354
- certmagic tags (version boundary v0.25.0 vulnerable -> v0.25.1 fixed) — https://api.github.com/repos/caddyserver/certmagic/tags
- Caddy `go.mod` at **v2.8.4** (pins vulnerable certmagic) — https://raw.githubusercontent.com/caddyserver/caddy/v2.8.4/go.mod
- Caddy `go.mod` at **v2.11.0** (first Caddy pinning certmagic >= v0.25.1) — https://raw.githubusercontent.com/caddyserver/caddy/v2.11.0/go.mod
- Caddy issue **#7366** — empty-chain panic reported against Caddy — https://github.com/caddyserver/caddy/issues/7366
- certmagic issue **#152** — Pebble per-launch CA regeneration / account volatility (feeds the trigger) — https://github.com/caddyserver/certmagic/issues/152

Honest gaps (UNCONFIRMED — method to obtain stated inline): the exact per-attempt failure ordering in the failing run (method: enable Caddy JSON logging at debug and capture the two `ObtainCertificate` attempts' error classifications). Does not affect the source-proven mechanism or the version boundary.